

CHAPTER II. AUTOMATA HỮU HẠN VÀ NGÔN NGỮ CHÍNH QUY

§1. AUTOMATA HỮU HẠN

Trong chương I đã giới thiệu về các khái niệm cơ bản của mô hình tính toán, các mô tả về automata ở mức đơn giản và chưa chính thức. Tại thời điểm này, chúng ta chỉ biết automata là gì và nó có thể được biểu diễn bằng biểu đồ ra sao. Để tiếp tục, chúng ta phải mô tả một cách rõ ràng hơn, đưa ra định nghĩa chính thức và phát triển các kết quả đúng đắn. Chúng ta bắt đầu với một trường hợp đơn giản của biểu đồ tổng quát được giới thiệu ở chương trước, một “**accepter hữu hạn**”.

Một automata hữu hạn làm việc theo thời gian rời rạc như tất cả các mô hình tính toán khác. Automata đơn giản này không có bộ nhớ tạm thời nên bị hạn chế về khả năng “ghi nhớ” trong quá trình tính toán. Lượng thông tin hữu hạn được lưu trữ lại trong bộ điều khiển bằng cách đặt bộ điều khiển ở một trạng thái cụ thể. Vì số lượng các trạng thái là hữu hạn, một automata ở mỗi thời điểm, chỉ xử lý được lượng thông tin đầu ra chỉ phụ thuộc vào thông tin vào lúc đó. Các thiết bị như vậy là mô hình của các mạch tổ hợp.

I. Bộ chấp nhận hữu hạn đơn định

Cũng tương tự như tất cả các automata, bộ chấp nhận đơn định cũng có bộ điều khiển trạng thái bên trong, các quy tắc chuyển từ trạng thái này sang trạng thái khác, một số đầu và cách đưa ra quyết định chấp nhận xâu.

1. Định nghĩa

Định nghĩa 1.1.1.

-
-
-
-
-

Cơ chế

- Tại

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Dưới

- Nếu

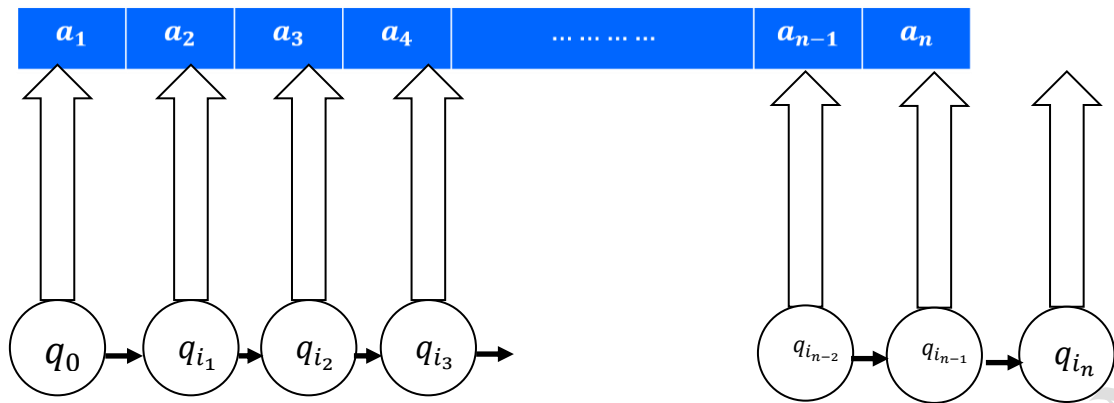
Quá trình đọc sâu u tiếp tục đến khi gặp một trong các trường hợp sau:

- DFA

- DFA

- DFA

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success



Chú ý.

- ✓ Tập Q thể hiện các trạng thái ghi nhớ của DFA trong quá trình đoán nhận, vì vậy khả năng ghi nhớ của DFA là hữu hạn. Mặt khác hàm chuyển δ đơn trị nên bước chuyển của automata luôn xác định một cách duy nhất.
- ✓ Nếu $D = Q \times \Sigma$ thì DFA luôn đọc xong xâu u và DFA được gọi là DFA đầy đủ.

Ví dụ 1.1.2. Cho DFA $M = (\{q_0, q_1, q_2\}, \{a, b\}, \delta, q_0, \{q_2\})$ trong đó hàm chuyển trạng thái δ xác định như sau: $\delta(q_0, a) = \delta(q_1, a) = \delta(q_2, a) = q_2$, $\delta(q_0, b) = \delta(q_1, b) = q_1$, $\delta(q_2, b) = \delta(q_3, a) = \delta(q_3, b) = q_2$.

a) Vẽ mô hình M đoán nhận các xâu $u_1 = bbaabab$, $u_2 = bbbbbaaa$.

--	--	--	--	--	--	--

Xâu u_1

--	--	--	--	--	--	--

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Xâu u_2

b) Viết dãy suy dẫn M đoán nhận các xâu $u_1 = bbaabab, u_2 = bbbbaaa$.

$q_0 u_1 =$

$q_0 u_2 =$

c) Lập bảng chuyển trạng thái và vẽ đồ thị biểu diễn M .

δ	a	b

d) Tìm $L(M)$ và viết biểu thức chính quy mô tả $L(M)$.

Để

- Đường đi

với

- Đường đi

với

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Biểu thức

e) Xây dựng văn phạm chính quy G tương đương với M . Cho G sinh ra các chuỗi u_1, u_2 .

Do

Do

Do

Do

Do

Do

Do

Vậy

q_0

nên

q_0

Để tiện lợi ta sẽ sử dụng hàm chuyển trạng thái mở rộng từ $\delta^*: Q \times \Sigma^* \rightarrow Q$. Khi đó đối số thứ hai của hàm δ^* là một chuỗi thuộc Σ^* chứ không phải là một ký hiệu đơn lẻ thuộc δ và giá trị của hàm δ^* cho chúng ta biết trạng thái của automata sau khi đọc chuỗi này.

Ví dụ 1.1.3. Nếu $\delta(q_0, 1) = q_1, \delta(q_1, 0) = q_2$ thì ta viết $\delta^*(q_0, 10) = q_2$.

Về mặt hình thức, ta có thể định nghĩa hàm chuyển trạng thái mở rộng δ^* như sau:

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

$$\delta^*(q, \lambda) = q$$

$$\delta^*(q, wa) = \delta(\delta^*(q, w), a)$$

với mọi $q \in Q, a \in \Sigma$ và $w \in \Sigma^*$.

Ví dụ 1.1.4. Trong ví dụ 1.1.3 ta có:

$$\delta^*(q_0, 10) = \delta(\delta^*(q_0, 1), 0) = \delta(\delta(\delta^*(q_0, \lambda), 1), 0) = \delta(\delta(q_0, 1), 0) = \delta(q_1, 0) = q_2$$

2. Ngôn ngữ đoán nhận bởi DFA

Định nghĩa 1.1.5.

$q_0 u =$

trong đó

Định nghĩa 1.1.6.

Định nghĩa 1.1.7.

3. Ý nghĩa của DFA

DFA đóng vai trò là mô hình tiêu chuẩn để máy tính thực thi các tác vụ nhận diện mẫu vì những lý do sau:

- **Tính dự đoán tuyệt đối:** Với một chuỗi đầu vào sw , DFA luôn đi qua một lộ trình trạng thái duy nhất. Điều này loại bỏ hoàn toàn sự mơ hồ và việc phải "thử sai" (backtracking).
- **Hiệu năng tối ưu ($O(n)$):** DFA chỉ cần đọc chuỗi đầu vào đúng một lần từ trái sang phải. Thời gian xử lý tỉ lệ thuận trực tiếp với độ dài chuỗi, không phụ thuộc vào độ phức tạp của mẫu (pattern).
- **Tiết kiệm tài nguyên:** Vì tại mỗi thời điểm máy chỉ ở đúng một trạng thái, bộ nhớ cần thiết để chạy một DFA là cực kỳ thấp (chỉ cần một biến để lưu ID trạng thái hiện tại).

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

4. Ứng dụng của DFA

DFA xuất hiện ở mọi nơi mà tốc độ và sự chính xác là ưu tiên hàng đầu:

a) Phân tích từ vựng trong Trình biên dịch (Lexical Analysis)

Đây là ứng dụng kinh điển nhất. Khi bạn viết code (C++, Python...), bước đầu tiên của trình biên dịch là biến các chuỗi ký tự thành các "Token" (từ khóa, tên biến, toán tử).

Ví dụ 1.1.8. Nhận diện từ khóa int, if, while. Các bộ quét từ vựng (như Lex hoặc Flex) sẽ chuyển các quy tắc này thành một DFA khổng lồ để quét mã nguồn với tốc độ chóng mặt.

b) Thiết kế phần cứng và Mạch số

Các kỹ sư điện tử sử dụng DFA dưới tên gọi Máy trạng thái hữu hạn (FSM) để điều khiển các thiết bị:

- × *Bộ điều khiển đèn giao thông*: Trạng thái chuyển từ Xanh → Vàng → Đỏ dựa trên bộ đếm thời gian.
- × *Máy bán hàng tự động*: Trạng thái thay đổi dựa trên số tiền bạn nạp vào cho đến khi đủ để xuất hàng.
- × *CPU*: Các đơn vị điều khiển bên trong chip xử lý thực chất là các DFA điều phối việc lấy lệnh và thực thi lệnh.

c) Giao thức truyền tin (Network Protocols)

Các giao thức mạng như TCP hoạt động dựa trên mô hình DFA để đảm bảo sự ổn định của kết nối:

- × Khi bạn gửi yêu cầu kết nối, máy chuyển từ trạng thái CLOSED sang SYN_SENT.
- × Nếu nhận được phản hồi, nó chuyển sang ESTABLISHED.
- × Mọi sự kiện (nhận gói tin, timeout) đều được quản lý chặt chẽ bởi các bước chuyển trạng thái của DFA.

d) Hệ thống lọc và tìm kiếm (String Matching)

- × *Công cụ grep trong Linux*: Khi bạn tìm kiếm một từ trong file văn bản lớn, grep thường chuyển biểu thức tìm kiếm của bạn thành DFA để đảm bảo tốc độ quét nhanh nhất có thể.
- × *Kiểm duyệt nội dung*: Các hệ thống lọc từ cấm trên mạng xã hội hoặc lọc thư rác thường dùng DFA để đối soát hàng triệu tin nhắn mỗi giây.

II. Bộ chấp nhận hữu hạn không đơn định

1. Định nghĩa và ngôn ngữ được đoán nhận

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định nghĩa 1.2.1.

-
-
-
-
-

Định nghĩa *NFA* khác với *DFA* ở ba điểm sau:

- Giá trị của hàm chuyển trạng thái không đơn thuần là một phần tử của Q mà là một tập con của Q . Tập hợp con này xác định tập hợp tất cả các trạng thái có thể đến được bởi hàm chuyển.
- Cho phép λ là đối số thứ hai của hàm chuyển trạng thái δ . Điều này có nghĩa là *NFA* có thể thực hiện bước chuyển mà không cần sử dụng đầu vào. Mặc dù chúng ta vẫn giả định rằng đầu đọc chỉ có thể di chuyển sang bên phải, nhưng có thể nó đứng yên trên một số bước di chuyển.
- Cuối cùng, trong một *NFA*, tập $\delta(q, a)$ có thể trống, nghĩa là không có sự chuyển đổi nào được xác định cho tình huống cụ thể này.

Nhận xét. *DFA* là một trường hợp đặc biệt của *NFA* vì $Q \in 2^Q$.

Định nghĩa 1.2.2.

Ví dụ 1.2.3. Cho $FAM = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$ trong đó $\delta(q_0, 0) = \{q_0, q_1\}$, $\delta(q_0, 1) = \{q_0\}$, $\delta(q_1, 1) = \{q_1, q_2\}$, $\delta(q_2, 0) = \delta(q_2, 1) = \{q_2\}$.

a) Chứng minh M là *NFA*.

M là

-

-

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) Cho M đoán nhận xâu $u_1 = 1010110010$ và $u_2 = 100101101$.

$q_0 u_1 =$

$q_0 u_1 =$

$q_0 u_1 =$

Vậy

$q_0 u_2 =$

Vậy

c) Lập bảng chuyển trạng thái và vẽ đồ thị biểu diễn M .

δ	0	1

d) Lập bảng chuyển trạng thái và vẽ đồ thị biểu diễn M .

Để

tức là

Biểu thức

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

e) Tìm DFA M_1 tương đương với M . Vẽ đồ thị chuyển của M_1 .

δ_1	0	1

Do

Vậy

Đồ thị

f) Cho M_1 đoán nhận xâu u_1, u_2 .

$r_0 u_1 =$

nên

$r_0 u_2 =$

nên

g) Xây dựng văn phạm chính quy G tương đương với M và cho G sinh ra các xâu u_1, u_2 .

Do

Do

Do

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Vậy

q_0

Vậy

q_0

Vậy

2. Ý nghĩa của NFA

Trong khi DFA (Đơn định) dễ cài đặt cho máy tính, thì NFA lại có ý nghĩa to lớn về mặt tư duy và thiết kế:

- Sự đơn giản trong thiết kế: NFA cho phép máy tính ở nhiều trạng thái cùng một lúc hoặc đi theo nhiều nhánh "dự đoán". Điều này giúp sơ đồ NFA thường nhỏ gọn và dễ hiểu hơn DFA rất nhiều lần đối với cùng một ngôn ngữ.
- Mô hình hóa sự "Dự đoán" (Guessing): NFA đại diện cho tư duy: "Nếu có một đường đi dẫn đến đích, tôi sẽ chấp nhận". Nó giúp giải quyết các bài toán mà chúng ta không biết chắc chắn lựa chọn nào là đúng ngay từ đầu.
- Tính tương đương toán học: Một định lý quan trọng là Mọi NFA đều có thể chuyển đổi thành DFA. Điều này có nghĩa là con người cứ việc thiết kế bằng NFA cho dễ, rồi để thuật toán tự động chuyển nó sang DFA cho máy chạy.

3. Ứng dụng của NFA

NFA không chỉ là lý thuyết suông, nó nằm dưới "nắp ca-pô" của nhiều công nghệ bạn dùng hàng ngày:

a) Xây dựng bộ máy tìm kiếm Biểu thức chính quy (Regex Engine)

Đây là ứng dụng phổ biến nhất. Khi bạn viết một Regex phức tạp (như kiểm tra email hay số điện thoại), hầu hết các thư viện lập trình sẽ:

- × Chuyển Regex đó thành một NFA (thường dùng thuật toán Thompson).
- × Sau đó, máy tính sẽ mô phỏng NFA này bằng cách theo dõi tất cả các trạng thái có thể có cùng lúc khi đọc văn bản.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) Kiểm chứng phần mềm và hệ thống (Model Checking)

Trong các hệ thống quan trọng (như phần mềm điều khiển tên lửa, xe tự lái), người ta dùng NFA để mô hình hóa mọi khả năng có thể xảy ra của hệ thống. NFA giúp kiểm tra xem có bất kỳ "nhánh" nào dẫn đến lỗi nghiêm trọng hay không.

c) Phân tích giao thức mạng (Network Protocols)

Các giao thức truyền tin như TCP hoặc các hệ thống phát hiện xâm nhập (IDS) sử dụng NFA để theo dõi các gói tin. Vì các gói tin có thể đến sai thứ tự hoặc bị mất, NFA cho phép hệ thống duy trì nhiều kịch bản trạng thái để xử lý sự không chắc chắn này.

d) Thiết kế bộ phân tích từ vựng (Lexical Analyzers)

Trong các trình biên dịch (như GCC cho C++, hay trình thông dịch Python), NFA được dùng để gom nhóm các ký tự thành các từ khóa (keywords), tên biến (identifiers). Người ta viết quy tắc bằng NFA rồi mới dùng công cụ (như Lex/Flex) để nén nó thành DFA tối ưu cho tốc độ.

III. Biểu diễn bộ chấp nhận hữu hạn

Hàm chuyển trạng thái là bộ phận quan trọng của FA. Cho một FA cũng là cho một hàm chuyển trạng thái. Hàm chuyển trạng thái có thể biểu diễn bởi bảng chuyển hoặc đồ thị chuyển.

1. Biểu diễn FA bằng bảng chuyển trạng thái

Cho bộ chấp nhận hữu hạn $M = (\{q_0, q_1, \dots, q_m\}, \{a_1, a_2, \dots, a_n\}, \delta, q_0, F)$. Khi đó hàm chuyển trạng thái δ có thể cho bởi bảng trong đó hàng i cột j là ô trống nếu $\delta(q_i, a_j)$ không xác định.

Trạng thái	Ký hiệu vào			
	a_1	a_2		a_n
q_1	$\delta(q_1, a_1)$	$\delta(q_1, a_2)$		$\delta(q_1, a_n)$
q_2	$\delta(q_2, a_1)$	$\delta(q_2, a_2)$		$\delta(q_2, a_n)$
.....				
q_m	$\delta(q_m, a_1)$	$\delta(q_m, a_2)$		$\delta(q_m, a_n)$

Chú ý.

- ✓ Nếu FA M cho bảng chuyển trạng thái thì nếu chỉ rõ tập trạng thái kết thúc F thì sẽ xác định được FA.
- ✓ Bảng chuyển trạng thái cho một gợi ý về cấu trúc dữ liệu để mô tả FA.
- ✓ Một FA có thể dễ dàng hiện thực thành một chương trình máy tính nhờ một dãy lệnh if.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ý nghĩa của biểu diễn FA bằng bảng chuyển

- Tính đầy đủ và chính xác: Bảng bắt buộc bạn phải liệt kê mọi tổ hợp có thể giữa trạng thái và ký tự nhập vào. Điều này đảm bảo không có bước chuyển nào bị bỏ sót (đặc biệt quan trọng đối với DFA).
- Cấu trúc dữ liệu cho máy tính: Máy tính không "nhìn" được hình vẽ, nhưng nó cực kỳ giỏi xử lý ma trận và mảng. Bảng chuyển chính là một mảng hai chiều mà máy tính có thể tra cứu trong thời gian $O(1)$.

Ứng dụng của biểu diễn FA bằng bảng chuyển

- ✖ Lập trình và Thực thi: Khi bạn viết mã nguồn cho một bộ quét từ vựng (Lexer), bạn sẽ lưu bảng chuyển này vào bộ nhớ. Mỗi lần nhận một ký tự, máy chỉ việc tra bảng:

`Trạng_thái_mới = Bang[Trạng_thái_hiện_tại][Ký_tu_nhập]`

- ✖ Chuyển đổi NFA sang DFA: Các thuật toán như Subset Construction (Xây dựng tập con) thực hiện hoàn toàn trên bảng vì nó giúp quản lý các tập hợp trạng thái một cách hệ thống, tránh nhầm lẫn so với việc vẽ tay.

2. Biểu diễn FA bằng đồ thị chuyển trạng thái

Cho

•

•

Nhận xét.

- ❖ Quá trình đoán nhận chuỗi $u = a_1a_2\dots a_n$ chính là một đường đi từ đỉnh đầu q_0 đến đỉnh q_i qua các cạnh có nhãn lần lượt là $a_1, a_2, \dots, a_n$.
- ❖ $L(M)$ chính là tập hợp các chuỗi trên đường đi từ đỉnh đầu đến đỉnh kết thúc.

Hàm chuyển trạng thái mở rộng của NFA có thể xác định đệ quy tương tự như đối với DFA nhưng thường sẽ không rõ ràng. Một định nghĩa được đánh giá cao hơn là dựa vào đồ thị chuyển.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định nghĩa 1.3.1. Cho NFA $M = (Q, \Sigma, \delta, q_0, F)$. Hàm chuyển trạng thái mở rộng δ^* xác định như sau: Với mỗi $q_i, q_j \in Q, u \in \Sigma^*$ thì $\delta(q_i, u)$ chứa q_j nếu có một đường đi từ đỉnh nhãn q_i sang đỉnh có nhãn q_j với nhãn đường đi là u .

- Khi đó ngôn ngữ được đoán nhận bởi M là

$$L(M) = \{u \in \Sigma^* | \delta^*(q_0, u) \cap F \neq \emptyset\}$$

Ý nghĩa của biểu diễn FA bằng đồ thị :

- **Trực quan hóa dòng chảy:** Giúp con người dễ dàng hình dung lộ trình của một chuỗi ký tự từ trạng thái bắt đầu đến đích. Bạn có thể thấy ngay các vòng lặp và các nhánh rẽ.
- **Phát hiện lỗi logic nhanh:** Bằng mắt thường, ta có thể nhận ra các "trạng thái chết" (trạng thái không bao giờ đến được đích) hoặc các thiếu sót trong luồng xử lý.
- **Tính trừu tượng cao:** Thích hợp để thảo luận ý tưởng, thiết kế hệ thống ở mức khái niệm.

Ứng dụng của biểu diễn FA bằng đồ thị chuyển

- ✖ **Thiết kế giao diện (UI/UX):** Mô tả các trạng thái của màn hình (ví dụ: *Đang tải* $\rightarrow$ *Thành công* $\rightarrow$ *Hiển thị dữ liệu*).
- ✖ **Đào tạo và giáo dục:** Là công cụ tốt nhất để dạy về logic và lý thuyết tính toán.

3. Sự kết hợp giữa đồ thị chuyển và bảng chuyển

Trong quy trình làm việc của một kỹ sư Khoa học máy tính, hai công cụ này hỗ trợ cho nhau theo chu trình:

- **Đồ thị:** Dùng để phác thảo ý tưởng ban đầu (Human-friendly).
- **Bảng:** Dùng để chuẩn hóa logic và kiểm tra tính đầy đủ (Verification).
- **Mã nguồn:** Chuyển bảng vào mảng/hàm trong code để chạy (Machine-friendly).

IV. Sự tương đương giữa DFA và NFA

Việc chuyển đổi từ Máy tự động hữu hạn không đơn định (NFA) sang Máy tự động hữu hạn đơn định (DFA) là một quy trình kỹ thuật cốt lõi trong lý thuyết tính toán. Về mặt bản chất, đây là quá trình biến một mô hình có tính "dự đoán/lựa chọn" thành một mô hình "quyết định/thực thi".

1. Phương pháp chuyển từ NFA thành DFA

Định nghĩa 1.4.1.

Do mỗi DFA cũng là NFA nên lớp ngôn ngữ được đoán nhận bởi DFA sẽ được đoán nhận bởi NFA. Ta xét trường hợp ngược lại.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định lý 1.4.2.

Giả sử $M = (Q, \Sigma, \delta, q_0, F)$ đoán nhận L . Xây dựng DFA $M' = (Q', \Sigma, \delta', q'_0, F')$ như sau:

- $Q' = 2^Q$.
- F' là tập mọi trạng thái trong Q' có chứa một trạng thái cuối nào đó của M .
- Một phần tử trong Q' là tập hợp $\{q_1, q_2, \dots, q_k\}$ được ký hiệu là $[q_1, q_2, \dots, q_k]$ trong đó $q_i \in Q$.
- Trạng thái đầu $q'_0 = [q_0]$.
- Hàm chuyển trạng thái δ' được thành lập theo công thức:

$$\delta'([q_1, q_2, \dots, q_k], a) = \bigcup_{i=1}^k \delta(q_i, a)$$

Hệ quả 1.4.3.

Chú ý. Trong 2^Q thường có nhiều phần tử không thể có đường đi từ $[q_0]$ nên không cần đưa chúng vào Q' . Do đó nên xuất phát từ $[q_0]$ rồi từng bước thêm các trạng thái mới, nếu các trạng thái này là kết quả của hàm chuyển áp dụng lên các trạng thái trước đó.

Ví dụ 1.4.4.

- a) Xây dựng FA M đoán nhận ngôn ngữ trên bảng chữ $\{a, b\}$ có hậu tố là xâu abb .

Vậy

- b) Xây dựng DFA M_1 tương đương với M và vẽ đồ thị chuyển biểu diễn M_1 . Cho M_1 đoán nhận xâu $u = abbababb$.

δ_1	a	b

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Do

Vậy

Đề thi

$r_0 u =$

Vậy

c) Xây dựng văn phạm chính quy G tương đương với M . Cho G sinh ra chuỗi u .

Do

Do

Do

Do

Vậy

$q_0 =$

Vậy

Ví dụ 1.4.5.

a) Xây dựng FA M đoán nhận ngôn ngữ trên bảng chữ nhị phân có chứa chuỗi con 000.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Vậy

b) Xây dựng DFA M_1 tương đương với M và vẽ đồ thị chuyển biểu diễn M_1 . Cho M_1 đoán nhận chuỗi $u = 011000010$.

δ_1	0	1

Do

Vậy

Đồ thị

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

$r_0 u =$

Vậy

c) Xây dựng văn phạm chính quy G tương đương với M . Cho G sinh ra xâu u .

Do

Do

Do

Do

Do

Vậy

$q_0 \Rightarrow$

Vậy

Ví dụ 1.4.6.

a) Xây dựng FA M đoán nhận ngôn ngữ trên bảng chữ nhị phân luôn chứa xâu con 000 và nếu có xuất hiện ký hiệu 0 thì ký hiệu 0 này phải thuộc xâu con có đúng ba chữ số 0 và không chứa xâu con 0000.

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) Xây dựng DFA M_1 tương đương với M và vẽ đồ thị chuyển biểu diễn M_1 . Cho M_1 đoán nhận xâu $u = 11000010$ và $v = 000110001$.

δ_1	0	1

Do

Vậy

$r_0 u =$

Ở

$r_0 v =$

Vậy

Đồ thị

c) Xây dựng văn phạm chính quy G tương đương với M . Cho G sinh ra xâu u .

Do

Do

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Do

Do

Do

Do

Vậy

$q_0 \Rightarrow$

Vậy

2. Ý nghĩa của việc chuyển từ NFA sang DFA

Việc chuyển đổi từ NFA sang DFA mang lại ba ý nghĩa lớn về mặt toán học và kỹ thuật:

- **Chứng minh tính tương đương:** Quy trình này chứng minh rằng tính "không đơn định" không làm tăng sức mạnh tính toán của máy tự động hữu hạn. Bất kỳ ngôn ngữ nào được chấp nhận bởi một NFA thì cũng luôn có một DFA tương ứng chấp nhận nó.
- **Loại bỏ sự mơ hồ:** NFA cho phép tồn tại nhiều lựa chọn cho cùng một đầu vào, điều này khiến máy tính khó xử lý trực tiếp. Chuyển sang DFA giúp xác định duy nhất một lộ trình trạng thái, giúp quá trình kiểm tra chuỗi trở nên tường minh.
- **Tối ưu hóa tốc độ thực thi:** Trong khi NFA cần theo dõi nhiều nhánh trạng thái cùng lúc (gây tốn bộ nhớ và thời gian xử lý), DFA chỉ cần một lần quét duy nhất từ trái sang phải với độ phức tạp thời gian là $O(n)$.

3. Ứng dụng của việc chuyển từ NFA sang DFA trong khoa học máy tính

Quy trình chuyển đổi này (thường thông qua thuật toán Subset Construction) là "trái tim" của nhiều công cụ phần mềm hiện đại:

a) Xây dựng bộ máy xử lý Biểu thức chính quy (Regex Engine)

Đây là ứng dụng phổ biến nhất. Hầu hết các thư viện Regex (như trong Python, Java, hay công cụ grep) hoạt động theo quy trình:

- ✦ Người dùng viết Regex.
- ✦ Hệ thống chuyển Regex thành NFA bằng thuật toán Thompson (Dễ thiết kế máy).
- ✦ Hệ thống tiếp tục chuyển NFA thành DFA để tìm kiếm trên văn bản với tốc độ nhanh nhất.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

b) Phân tích từ vựng trong Trình biên dịch (Lexical Analysis)

Các trình biên dịch ngôn ngữ lập trình sử dụng ứng dụng này để nhận diện các từ khóa (tokens).

- ✖ Các quy tắc nhận diện từ khóa (như if, while, tên biến) được định nghĩa bằng các NFA riêng lẻ.
- ✖ Sau đó, tất cả các NFA này được gộp lại và chuyển thành một DFA duy nhất để quét toàn bộ file mã nguồn chỉ trong một lượt duy nhất.

c) Kiểm chứng hệ thống (Formal Verification)

Trong việc kiểm tra logic của các hệ thống an toàn (như phần mềm điều khiển máy bay hay thiết bị y tế):

- ✖ Các kịch bản lỗi thường được mô tả dưới dạng NFA để bao hàm mọi khả năng xảy ra.
- ✖ Việc chuyển sang DFA giúp các công cụ kiểm chứng tự động duyệt qua toàn bộ không gian trạng thái một cách hệ thống để đảm bảo không có nhánh nào dẫn đến thảm họa.

Quy trình ứng dụng thực tế:

Ý tưởng (Regex) → Mô hình hóa (NFA) → Chuyển đổi (Subset Construction)

→ Thực thi hiệu năng cao (DFA).

BÀI TẬP BUỔI 3

Bài 1. Cho FA $M =$

a) Cho M đoán nhận xâu

nên

b) Cho M đoán nhận xâu

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

nên

c) Cho M đoán nhận sâu

nên

d) Tìm ngôn ngữ do M đoán nhận

Bài 2. Đơn định FA $M =$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

§2. AUTOMATA HỮU HẠN VÀ NGÔN NGỮ CHÍNH QUY

I. Biểu thức chính quy

1. Định nghĩa và ví dụ

Biểu thức chính quy (Regular Expression - Regex) là một chuỗi các ký tự đặc biệt dùng để mô tả một bộ quy tắc tìm kiếm. Trong lý thuyết tính toán, Regex là cách biểu diễn đại số cho ngôn ngữ chính quy, có sức mạnh tương đương với máy tự động hữu hạn (FA).

Định nghĩa 2.1.1.

•

•

•

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ví dụ 2.1.2. Với bảng chữ $\Sigma = \{a, b\}$ thì $(ab + a)^*(a + \lambda)(aba + \emptyset)^*$ là biểu thức chính quy.

Biểu thức chính quy có thể được sử dụng để mô tả một số ngôn ngữ đơn giản. Nếu r là một biểu thức chính quy, chúng ta sẽ ký hiệu $L(r)$ là ngôn ngữ liên mô tả bởi r .

Định nghĩa 2.1.3.

•

•

•

•

•

Bốn quy tắc cuối cùng của định nghĩa này được sử dụng một cách đệ quy để đưa $L(r)$ về các thành phần đơn giản hơn. Quy tắc đầu tiên là điều kiện để kết thúc đệ quy này. Để biết một ngôn ngữ mà một biểu thức biểu thị, ta áp dụng các quy tắc này lặp đi lặp lại.

Ví dụ 2.1.4. Hãy biểu diễn các ngôn ngữ có biểu thức chính quy sau và mô tả ngôn ngữ này:

a) $(0 + 1)^*$

b) $a^*(a + b)$

c) $(a + b)^*aba$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

d) $(a + b)^*(a + ab)$

$$L((a + b)^*(a + ab)) = (L(a + b))^* L(a + ab) = \{a, b\}^* \{a, ab\}$$

Vậy ngôn ngữ do biểu thức chính quy $(a + b)^*(a + ab)$ mô tả các chuỗi có hậu tố a hoặc có hậu tố ab trên bảng chữ $\{a, b\}$.

e) $(0 + 01)^*$

f) $(00)^*(111)^*$

Ví dụ 2.1.5. Tìm các biểu thức chính quy mô tả các ngôn ngữ sau:

a) L_1 = tập các chuỗi nhị phân có chứa chuỗi con 01.

$L_1 =$

b) L_2 là tập các chuỗi có dạng $a^n b^m$ với $n \geq 3, m$ là số lẻ.

$L_2 =$

nên

2. Ý nghĩa của Biểu thức chính quy

- Sự ngắn gọn và súc tích: Một dòng Regex có thể thay thế cho hàng chục dòng lệnh if-else phức tạp. Nó cho phép bạn mô tả các cấu trúc văn bản lặp lại một cách cực kỳ gọn nhẹ.
- Tính trừu tượng hóa cao: Regex không quan tâm đến nội dung cụ thể mà quan tâm đến cấu trúc của dữ liệu. Ví dụ: Bạn không tìm số "090...", bạn tìm "một chuỗi bắt đầu bằng số 0 và có 10 chữ số".

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- **Cầu nối giữa con người và máy tính:** *Regex là ngôn ngữ "khai báo". Bạn chỉ cần nói cho máy biết bạn muốn tìm cái gì, còn việc tìm như thế nào (chạy qua các trạng thái DFA/NFA nào) là do bộ máy Regex tự xử lý.*
- **Tính chuẩn hóa:** *Hầu hết các ngôn ngữ lập trình (Python, Java, JavaScript, C#...) và các công cụ hệ thống (Linux, SQL) đều hỗ trợ Regex với cú pháp gần như tương đồng.*

3. Ứng dụng thực tế của Regex

a) Kiểm tra tính hợp lệ của dữ liệu (Data Validation)

Đây là ứng dụng phổ biến nhất trong lập trình web và ứng dụng.

Ví dụ 2.1.6. Kiểm tra xem người dùng có nhập đúng định dạng Email, số điện thoại, hoặc mật khẩu có đủ độ mạnh (bao gồm chữ hoa, chữ thường và ký số) hay không.

b) Tìm kiếm và thay thế nâng cao (Find & Replace)

Trong các trình soạn thảo văn bản (Notepad++, VS Code) hoặc các công cụ xử lý văn bản (Word): Bạn có thể tìm tất cả các ngày tháng có định dạng dd/mm/yyyy và chuyển chúng thành yyyy-mm-dd chỉ bằng một lệnh Replace duy nhất nhờ tính năng Capture Groups của Regex.

c) Phân tích từ vựng trong Trình biên dịch (Lexical Analysis)

Như đã thảo luận ở các phần trước, các trình biên dịch sử dụng Regex để nhận diện các "Token": Xác định đâu là từ khóa (if, else), đâu là hằng số số thực (3.14), đâu là tên biến. Regex là công cụ đầu tiên "đọc" mã nguồn của bạn.

d) Trích xuất dữ liệu (Data Scraping)

Khi cần lấy thông tin từ một đồng dữ liệu hỗn độn hoặc từ mã nguồn HTML của một trang web: Regex giúp lọc ra danh sách các liên kết (links), giá sản phẩm, hoặc mã chứng khoán từ các trang tin tức một cách tự động.

e) Quản trị hệ thống và An ninh mạng

- ✖ **Phân tích Log:** Các kỹ sư hệ thống dùng Regex để quét qua hàng triệu dòng log máy chủ nhằm tìm ra các dấu hiệu bị tấn công (như SQL Injection hoặc Brute Force).
- ✖ **Cấu hình Server:** Các máy chủ web như Nginx hoặc Apache dùng Regex để điều hướng (rewrite) URL, giúp tạo ra các đường dẫn thân thiện với người dùng và SEO.

4. Mối quan hệ giữa Regex và Automata

Để thực thi một biểu thức Regex, máy tính thường trải qua quy trình sau:

- Chuyển Regex sang NFA: Sử dụng thuật toán Thompson.
- Chuyển NFA sang DFA: Để tối ưu tốc độ tìm kiếm.
- Thực thi: Máy tính chạy chuỗi văn bản qua DFA đó để trả về kết quả Đúng/Sai.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

II. Ngôn ngữ chính quy

1. Định nghĩa và ví dụ

Định nghĩa 2.2.1.

Ví dụ 2.2.2. Chứng minh rằng ngôn ngữ $L = \{a^n b^m \mid n, m \geq 1\}$ là ngôn ngữ chính quy.

Do

nên

Định lý 2.2.3.

Ví dụ 2.2.4. Chỉ ra các NFA đoán nhận các ngôn ngữ có biểu thức chính quy mô tả là $\emptyset, \lambda, a, a(a+b)^*ab$ trên bảng chữ $\{a, b\}$.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định lý 2.2.5.

Ví dụ 2.2.6. Cho L tập các xâu nhị phân luôn chứa cặp 00 và có tiền tố là 01.

a) Chứng minh rằng L là ngôn ngữ chính quy.

L

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

L

b) Chỉ ra biểu thức chính quy r có biểu thức mô tả là L .

L

2. Ý nghĩa của việc định nghĩa ngôn ngữ chính quy thông qua FA

- **Tính khả thi về mặt tính toán (Computational Feasibility):** Một ngôn ngữ được gọi là "chính quy" nếu và chỉ nếu có một FA chấp nhận nó. Điều này có nghĩa là ngôn ngữ đó có thể được xử lý bởi một cỗ máy có bộ nhớ hữu hạn và cố định, không phụ thuộc vào độ dài chuỗi đầu vào.
- **Tính đơn định và hiệu quả:** Khi định nghĩa bằng DFA, ta có một quy trình xử lý cực kỳ nhanh với độ phức tạp thời gian là $O(n)$. Máy tính không bao giờ phải "đoán" hay quay lui, giúp việc kiểm tra một chuỗi có thuộc ngôn ngữ hay không trở nên tối ưu tuyệt đối.
- **Cơ sở để chứng minh các tính chất ngôn ngữ:** Nhờ FA, các nhà khoa học có thể sử dụng các công cụ toán học (như Bổ đề bơm - Pumping Lemma) để chứng minh một ngôn ngữ có chính quy hay không, hoặc chứng minh các tính chất đóng (hợp, giao, bù) của ngôn ngữ chính quy một cách dễ dàng.
- **Sự thống nhất giữa các mô hình:** FA cung cấp một "máy thực thi" cho các khái niệm khác. Biểu thức chính quy (Regex) là cách con người viết, còn FA là cách máy tính hiểu. Việc định nghĩa qua FA giúp kết nối đại số (Regex), văn phạm (Regular Grammar) và cơ học (Automata).

3. Ứng dụng thực tế

Khi một ngôn ngữ được định nghĩa và triển khai dưới dạng một FA, nó mở ra các ứng dụng thực tiễn sau:

a) Phân tích từ vựng trong trình biên dịch (Lexical Analysis)

Đây là ứng dụng quan trọng nhất. Các ngôn ngữ lập trình định nghĩa các "từ" của mình (như số thực, tên biến, từ khóa) là các ngôn ngữ chính quy.

Ví dụ. Một FA sẽ được thiết kế để nhận diện đâu là số thực hợp lệ (ví dụ: 3.14, -0.5). Nếu chuỗi đầu vào đi đến trạng thái kết thúc của FA, trình biên dịch sẽ xác nhận đó là một Token hợp lệ.

b) Thiết kế giao thức mạng và truyền thông

Các ngôn ngữ chính quy qua FA dùng để mô tả các quy tắc giao tiếp.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Ứng dụng: Trong giao thức TCP, các trạng thái như LISTEN, SYN_SENT, ESTABLISHED tạo thành một FA. Ngôn ngữ của FA này chính là tập hợp các trình tự gói tin hợp lệ. Nếu một chuỗi gói tin không tuân theo lộ trình của FA, nó sẽ bị coi là lỗi bảo mật hoặc lỗi kết nối.

c) Kiểm tra và xác thực dữ liệu (Input Validation)

Mọi hệ thống đăng ký web đều sử dụng FA (ẩn dưới dạng Regex) để định nghĩa "ngôn ngữ" của các trường nhập liệu.

Ứng dụng: Kiểm tra định dạng Email, mật khẩu, số thẻ tín dụng. FA giúp máy tính phản hồi ngay lập tức xem dữ liệu có khớp với mẫu hay không.

d) Thiết kế Hệ thống nhúng và Phần cứng

Trong các chip vi xử lý, các đơn vị điều khiển thực chất là các FA phần cứng.

Ứng dụng: Điều khiển thang máy, máy giặt, hay các mạch logic trong CPU. "Ngôn ngữ" ở đây là các chuỗi tín hiệu điện đầu vào, và FA đảm bảo hệ thống phản ứng chính xác với từng chuỗi tín hiệu đó.

e) Xử lý văn bản và Tìm kiếm (Pattern Matching)

Các bộ lọc thư rác (Spam filters) hoặc công cụ tìm kiếm sử dụng FA để quét nhanh qua hàng tỷ ký tự.

Ứng dụng: Tìm kiếm các mẫu virus trong file (Antivirus), lọc các từ nhạy cảm trên mạng xã hội. Vì FA xử lý theo thời gian thực (linear time), nó cực kỳ hiệu quả cho các hệ thống dữ liệu lớn.

III. Văn phạm chính quy phải và ngôn ngữ chính quy

Mối liên hệ giữa văn phạm chính quy và Automata hữu hạn là một trong những cột trụ quan trọng nhất của lý thuyết tính toán. Về cơ bản, chúng là hai mặt của một đồng xu: một bên là cách tạo ra ngôn ngữ và một bên là cách nhận diện ngôn ngữ đó.

1. Văn phạm chính quy và automata hữu hạn

Định lý 2.3.1.

Xây dựng NFA $M = (Q, T, \delta, q_0, F)$ đoán nhận ngôn ngữ $L(G)$ như sau:

- Thêm vào ký hiệu mới E không thuộc $V \cup T$.
- $q_0 = S$.
- $F = \{E\}$ nếu không có sản xuất $S \rightarrow \lambda$ và $F = \{S, E\}$ nếu có sản xuất $S \rightarrow \lambda$.
- Nếu $A \rightarrow uB$ là một sản xuất trong P với $u = a_1 a_2 \dots a_k$ với $A, B \in V, u \in T^*$ thì thêm vào $k - 1$ trạng thái mới $\overline{a_1}, \overline{a_2}, \dots, \overline{a_{k-1}}$ và $\delta(A, a_1) = \overline{a_1}, \delta(\overline{a_1}, a_2) = \overline{a_2}, \dots, \delta(\overline{a_{k-2}}, a_{k-1}) = \overline{a_{k-1}}, \delta(\overline{a_{k-1}}, a_k) = B$.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- Nếu $A \rightarrow u$ là một sản xuất trong P với $u = a_1 a_2 \dots a_k$ với $A, B \in V, u \in T^*$ thì thêm vào $k - 1$ trạng thái mới $\overline{a_1}, \overline{a_2}, \dots, \overline{a_{k-1}}$ và $\delta(A, a_1) = \overline{a_1}, \delta(\overline{a_1}, a_2) = \overline{a_2}, \dots, \delta(\overline{a_{k-2}}, a_{k-1}) = \overline{a_{k-1}}, \delta(\overline{a_{k-1}}, a_k) = E$.
- $Q = V \cup \{E\} \cup \{\text{các trạng thái mới thêm vào}\}$

Khi đó ta có thể chứng minh được M đoán nhận ngôn ngữ $L(G)$.

Ví dụ 2.3.2. Cho văn phạm chính quy $G = (\{S, A\}, \{0,1\}, S, \{S \rightarrow 0A, A \rightarrow 01S|1\})$

a) Tìm $L(G)$ và biểu thức chính quy r mô tả ngôn ngữ $L(G)$.

S

Vậy

b) Xây dựng NFA M tương đương với G và vẽ đồ thị chuyển của M .

S

A

A

Vậy

Đồ thị

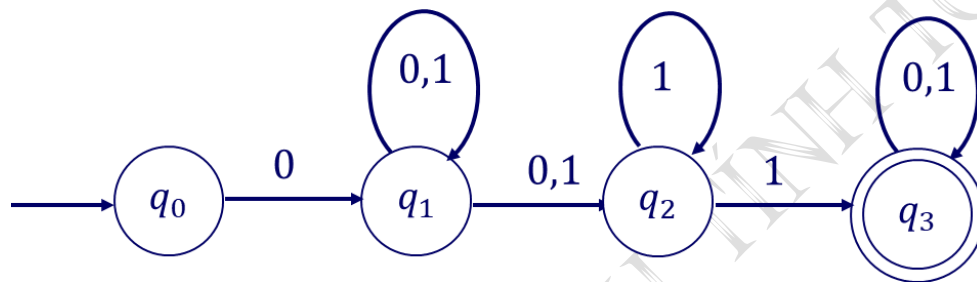
Định lý 2.3.3.

Giả sử $M = (\{q_0, q_1, \dots, q_n\}, \{a_1, a_2, \dots, a_m\}, \delta, q_0, F)$ là một DFA đoán nhận L . Xây dựng văn phạm chính quy phải $G = (\{q_0, q_1, \dots, q_n\}, \{a_1, a_2, \dots, a_m\}, q_0, P)$ sinh ra ngôn ngữ L với tập sản xuất P xây dựng như sau:

- Với mỗi $\delta(q_i, a_j) = q_k$, thêm vào P sản xuất $q_i \rightarrow a_j q_k$.
- Nếu $q_k \in F$ thì thêm vào P một sản xuất nữa $q_i \rightarrow \lambda$.

Khi đó có thể chứng minh được $L(G) = L(M)$ hay G sinh ra ngôn ngữ L .

Ví dụ 2.3.4. Cho FA M có đồ thị chuyển như sau:



a) Mô tả $L(M)$ và viết biểu thức chính quy r mô tả ngôn ngữ $L(M)$.

Biểu thức

b) Xây dựng văn phạm chính quy G sinh ra ngôn ngữ $L(M)$.

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định lý 2.3.5.

Ví dụ 2.3.6. Tìm văn phạm chính quy G sinh ra ngôn ngữ $L = \{0^n 1^m \mid n + m \text{ là số chẵn}\}$.

Trường hợp $n = 0, m = 0$

Trường hợp n chẵn, $m = 0$

Trường hợp n lẻ, $m = 1$

Trường hợp $n = 0, m = 2$

Trường hợp $n = 0, m$ chẵn

Kết hợp

Xây dựng

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Vậy

2. Ý nghĩa mối liên hệ giữa văn phạm chính quy và FA

a) Sự tương đương hoàn toàn (Mathematical equivalence)

Ý nghĩa lớn nhất là tính tương đương: Một ngôn ngữ là chính quy nếu và chỉ nếu nó có thể được mô tả bằng Văn phạm chính quy và được chấp nhận bởi một FA.

- Văn phạm chính quy (RG): Đóng vai trò là bộ quy tắc "sinh". Nó giống như công thức nấu ăn, hướng dẫn bạn cách lắp ghép các ký tự để tạo thành một chuỗi đúng.
- Automata hữu hạn (FA): Đóng vai trò là bộ máy "kiểm chứng". Nó giống như một nhân viên kiểm định, đứng ở cuối dây chuyền để xem sản phẩm có đúng công thức không.

b) Chuyển đổi từ Tư duy sang Thuật toán

Con người thường tư duy theo quy tắc (văn phạm), nhưng máy tính cần các bước chuyển trạng thái cụ thể (Automata). Mối liên hệ này cho phép chúng ta:

- Thiết kế quy tắc bằng ngôn ngữ tự nhiên/văn phạm.
- Tự động hóa việc chuyển quy tắc đó thành sơ đồ trạng thái.
- Cài đặt sơ đồ đó vào phần cứng hoặc phần mềm.

3. Ứng dụng thực tế

a) Trình phân tích từ vựng (Lexical Analyzer/Scanner)

Trong các trình biên dịch như GCC hay Clang, mối liên hệ này được sử dụng triệt để:

- ✖ Văn phạm: Định nghĩa thể nào là một "số nguyên", "tên biến" hay "từ khóa".
- ✖ FA: Khi bạn nhấn "Compile", trình biên dịch sẽ chạy một FA được xây dựng từ văn phạm đó để quét qua mã nguồn của bạn. Nếu FA rơi vào trạng thái lỗi, bạn sẽ nhận được thông báo "Syntax Error".

b) Chế tạo công cụ sinh mã tự động (Scanner Generators)

Các công cụ nổi tiếng như Lex hay Flex hoạt động dựa trên mối liên hệ này.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

- * Lập trình viên chỉ cần viết các văn phạm chính quy (dưới dạng Regex).
- * Công cụ sẽ tự động áp dụng các định lý toán học để xuất ra mã nguồn C/C++ chứa một DFA tối ưu dùng để nhận diện các mẫu đó.

c) Xử lý ngôn ngữ tự nhiên (NLP) ở mức cơ bản

Trong các hệ thống phân tích văn bản đơn giản:

- * Người ta dùng văn phạm chính quy để mô tả cấu trúc câu hoặc từ vựng.
- * FA được dùng để trích xuất thông tin (Information Extraction) với tốc độ cực nhanh mà không cần đến các mô hình AI phức tạp tốn tài nguyên.

d) Kiểm chứng và Đặc tả hệ thống

Trong kỹ thuật phần mềm, các yêu cầu hệ thống thường được viết dưới dạng quy tắc (Grammar). Việc chuyển chúng sang FA giúp các kỹ sư chạy mô phỏng để xem các quy tắc đó có mâu thuẫn nhau không, hoặc có dẫn đến "trạng thái chết" (Deadlock) nào không trước khi bắt đầu lập trình.

4. Tại sao mối liên hệ giữa văn phạm chính quy và FA quan trọng

Nếu không có mối liên hệ này, việc viết các chương trình kiểm tra chuỗi sẽ cực kỳ thủ công và dễ sai sót. Thay vì viết hàng trăm câu lệnh *if – else* lồng nhau:

- Định nghĩa quy tắc bằng văn phạm.
- Hệ thống (thư viện ngôn ngữ như *re* trong Python, *java.util.regex* trong Java) sẽ tự động xây dựng một FA ngầm định bên dưới.
- Kết quả là code của ngắn gọn hơn, dễ bảo trì hơn và hiệu suất cực kỳ cao (độ phức tạp $O(n)$).

IV. Tính chất của ngôn ngữ chính quy

1. Automata hữu hạn không xuất phát lại

Định nghĩa 2.4.1.

Định lý 2.4.2.

Giả sử DFA $M = (\{q_0, q_1, \dots, q_n\}, \Sigma, \delta, q_0, F)$. Xây dựng FA $M' = (Q \cup \{q_{n+1}\}, \Sigma, \delta', q_0, F')$ như sau:

$$\delta'(q, a) = \begin{cases} \delta(q, a) & \text{khi } \delta(q, a) \neq q_0 \\ q_{n+1} & \delta(q, a) = q_0 \end{cases} \text{ với } q \in Q$$
$$\delta(q_{n+1}, a) = \delta(q_0, a)$$

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

$$F' = \begin{cases} F & \text{nếu } q_0 \notin F \\ F \cup \{q_{n+1}\} & \text{nếu } q_0 \in F \end{cases}$$

M' thực hiện các bước chuyển hoàn toàn giống M trừ khi M chuyển về trạng thái q_0 thì M' chuyển về trạng thái q_{n+1} .

Ví dụ 2.4.3. Xây dựng FA M không xuất phát lại đoán nhận các xâu trên bảng chữ $\{a, b\}$ có hậu tố bab .

Bước 1.

Bước 2.

δ_2	a	b

Do

Vậy

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Bước 3.

M

2. Các phép toán trên ngôn ngữ chính quy

Định lý 2.4.4.

Giả sử L_1, L_2 là hai ngôn ngữ chính quy được đoán nhận bởi các DFA không xuất phát lại $M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$ và $M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$ với $Q_1 \cap Q_2 = \emptyset$. Xây dựng FA $M = (Q, \Sigma, \delta, q_0, F)$ đoán nhận $L_1 \cup L_2$ như sau:

$$\begin{aligned} Q &= Q_1 \cup Q_2 \cup \{q_0\} \setminus \{q_{01}, q_{02}\} \\ F &= \begin{cases} F_1 \cup F_2 \cup \{q_0\} \setminus \{q_{01}, q_{02}\} & \text{nếu } q_{01} \in F_1 \text{ hoặc } q_{02} \in F_2 \\ F_1 \cup F_2 & \text{nếu } q_{01} \notin F_1 \text{ hoặc } q_{02} \notin F_2 \end{cases} \\ \delta(q, a) &= \begin{cases} \delta_1(q, a) & \text{nếu } q \in Q_1 \setminus \{q_{01}\} \\ \delta_2(q, a) & \text{nếu } q \in Q_2 \setminus \{q_{02}\} \end{cases} \\ \delta(q_0, a) &= \delta_1(q_{01}, a) \cup \delta_2(q_{02}, a) \end{aligned}$$

Do $Q_1 \cap Q_2 = \emptyset$ và M_1, M_2 là các DFA không xuất phát lại nên mỗi khi ở bước thứ nhất trong M thực hiện thì M sẽ bắt chước bước chuyển trạng thái đầu tiên hoặc của M_1 hoặc của M_2 rồi sau đó thực thi giống như M_1 hoặc M_2 . Vậy $L(M) = L_1 \cup L_2$.

Ví dụ 2.4.5. Xây dựng FA M đoán nhận các xâu nhị phân hoặc có tiền tố 11 hoặc có hậu tố 11.

Bước 1.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Bước 2.

Bước 3.

δ_3	a	b

Do

Vậy

Bước 4.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Bước 5.

Định lý 2.4.6. Nếu L là ngôn ngữ chính quy thì $\Sigma^* \setminus L$ cũng là ngôn ngữ chính quy.

Giả sử L được đoán nhận bởi $M = (Q, \Sigma, \delta, q_0, F)$. Xây dựng $M' = (Q, \Sigma, \delta, q_0, F')$ với $F' = Q \setminus F$.

Khi đó M' hoạt động giống hệt như M , chỉ khác do $F' = Q \setminus F$ nên các chuỗi M đoán nhận thì M' không đoán nhận còn các chuỗi M không đoán nhận thì M' đoán nhận.

Ví dụ 2.4.7. Xây dựng DFA M đoán nhận các chuỗi trên bảng chữ $\{a, b\}$ không có hậu tố là $babb$.

Bước 1.

Bước 2.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định lý 2.4.8. Nếu L_1, L_2 là các ngôn ngữ chính quy trên bảng chữ Σ_1, Σ_2 thì $L_1 \cap L_2$ cũng là ngôn ngữ chính quy.

Theo công thức De-Morgan: $L_1 \cap L_2 = \overline{\overline{L_1} \cup \overline{L_2}} = \overline{\Sigma^* \setminus ((\Sigma^* \setminus L_1) \cup (\Sigma^* \setminus L_2))}$ nên $L_1 \cap L_2$ là ngôn ngữ chính quy.

Hệ quả 2.4.9. Lớp các ngôn ngữ chính quy tạo thành một đại số.

Hệ quả 2.4.10. Nếu L là một ngôn ngữ chính quy trên bảng chữ Σ và $\Sigma \subset \Sigma'$. Khi đó có một FA với bảng chữ vào Σ đoán nhận L khi và chỉ khi có một FA với bảng chữ vào Σ' đoán nhận L .

Định lý 2.4.11.

Giả sử L_1, L_2 là hai ngôn ngữ chính quy được đoán nhận bởi các DFA $M_1 = (Q_1, \Sigma_1, \delta_1, q_{01}, F_1)$ và $M_2 = (Q_2, \Sigma_2, \delta_2, q_{02}, F_2)$ với $Q_1 \cap Q_2 = \emptyset$.

Xây dựng FA $M = (Q, \Sigma, \delta, q_0, F)$ đoán nhận $L_1 L_2$ như sau:

$$Q = Q_1 \cup Q_2$$

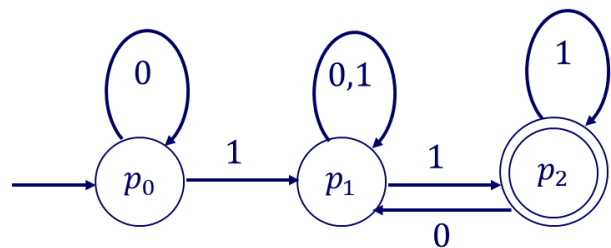
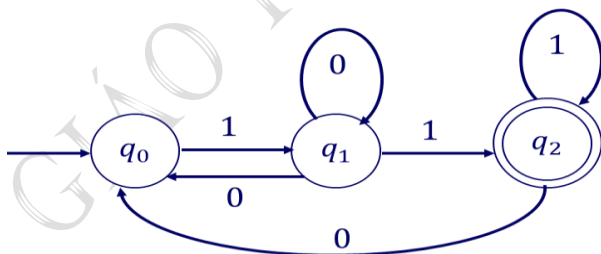
$$\Sigma = \Sigma_1 \cup \Sigma_2$$

$$\delta(q, a) = \begin{cases} \{\delta_1(q, a)\} & \text{nếu } q \in Q_1 \setminus F_1, a \in \Sigma_1 \\ \{\delta_1(q, a), \delta_2(q_{02}, a)\} & \text{nếu } q \in F_1, a \in \Sigma_1 \\ \{\delta_2(q, a)\} & \text{nếu } q \in Q_2, a \in \Sigma_2 \end{cases}$$

$$F = \begin{cases} F_2 & \text{nếu } \lambda \notin L_2 \\ F_1 \cup F_2 & \text{nếu } \lambda \in L_2 \end{cases}$$

Ở phần tiền tố, M hoạt động giống hệt như M_1 . Sau khi đến trạng thái kết thúc của M_1 thì hoặc M tiếp tục bắt chước M_1 hoặc chuyển sang bắt chước M_2 , do đó M đoán nhận ngôn ngữ $L_1 L_2$ nên $L_1 L_2$ là ngôn ngữ chính quy.

Ví dụ 2.4.12. Xây dựng DFA M đoán nhận ngôn ngữ $L_1 L_2$ cho bởi các FA M_1, M_2 sau:



Bước 1.

δ_3	0	1

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Do

Vậy

Bước 2.

Định lý 2.4.13.

Giả sử L được đoán nhận bởi DFA không xuất phát lại $M = (Q, \Sigma, \delta, q_0, F)$. Xây dựng FA $M' = (Q, \Sigma, \delta', q_0, F')$ đoán nhận L^* như sau:

$$\delta'(q, a) = \begin{cases} \{\delta(q, a)\} & \text{nếu } \delta(q, a) \notin Q \setminus F \\ \{\delta(q, a), q_0\} & \text{nếu } \delta(q, a) \in F \end{cases}$$

$$F' = F \cup \{q_0\}$$

M' luôn bắt chước M nhưng mỗi khi rơi vào trạng thái kết thúc thì hoặc M' cũng về trạng thái kết thúc hoặc quay về q_0 . Do đó $L(M') = L^*$ nên L^* là ngôn ngữ chính quy.

Ví dụ 2.4.14. Xây dựng FA M đoán nhận ngôn ngữ trên bảng chữ $\{a, b\}$ đoán nhận ngôn ngữ chính quy có biểu thức chính quy biểu diễn là $(bab + aba)^*$.

Bước 1.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Bước 2.

Bước 3.

Bước 4.

V. Định lý Kleene

1. Định lý

Định lý Kleene, công bố năm 1956 bởi Stephen Kleene, được coi là định lý quan trọng nhất trong lý thuyết automata. Nó thiết lập một mối quan hệ tương đương tuyệt đối giữa các mô hình toán học khác nhau của ngôn ngữ chính quy.

Định lý 2.5.1. $\emptyset$ và $\{\lambda\}$ là các ngôn ngữ chính quy.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Định lý 2.5.2. $\forall u \in \Sigma^*$ thì $\{u\}$ là ngôn ngữ chính quy.

Hệ quả 2.5.3. Mọi tập con hữu hạn của Σ^* đều là ngôn ngữ chính quy.

Hệ quả 2.5.4. Mọi ngôn ngữ tạo nên từ ngôn ngữ hữu hạn bằng cách áp dụng một số hữu hạn lần các phép hợp, ghép tiếp, bao đóng đều là ngôn ngữ chính quy.

Định lý 2.5.5 (Định lý Kleene). Mọi ngôn ngữ chính quy đều có thể nhận được từ các ngôn ngữ hữu hạn bằng cách áp dụng một số hữu hạn lần các phép hợp, ghép tiếp, bao đóng.

2. Ý nghĩa của định lý Kleene

Ý nghĩa lớn nhất của định lý này là khả năng hợp nhất các cách tiếp cận khác nhau để mô tả cùng một vấn đề:

- **Tính tương đương:** Một ngôn ngữ là chính quy nếu và chỉ nếu nó có thể được mô tả bằng một biểu thức chính quy, hoặc được chấp nhận bởi một DFA, hoặc được chấp nhận bởi một NFA.
- **Sự tự do trong biểu diễn:** Định lý cho phép chúng ta chọn công cụ phù hợp nhất cho từng giai đoạn. Con người có thể dùng Regex để mô tả (vì nó ngắn gọn), nhưng máy tính sẽ dùng DFA để thực thi (vì nó nhanh).
- **Tính đóng:** Định lý Kleene giúp chứng minh các tính chất quan trọng của ngôn ngữ chính quy (như phép hợp, phép giao, phép lấy bù) một cách dễ dàng thông qua việc thao tác trên sơ đồ máy tự động.

3. Hai chiều của định lý Kleene

Định lý này gồm hai phần chứng minh tương ứng với hai chiều chuyển đổi:

a) Chiều thuận: Từ biểu thức chính quy (RE) sang Automata (FA)

Chiều này chứng minh rằng mọi quy tắc mô tả bằng Regex đều có thể "chế tạo" thành một cái máy để chạy.

- Thuật toán tiêu biểu: Thuật toán Thompson.
- Ý nghĩa: Đây là cơ sở để các phần mềm (như trình duyệt, IDE) hiểu được các chuỗi Regex bạn viết.

b) Chiều nghịch: Từ Automata (FA) sang biểu thức chính quy (RE)

Chiều này chứng minh rằng bất kỳ hệ thống máy móc phức tạp nào (có trạng thái hữu hạn) đều có thể được tóm tắt lại bằng một dòng công thức Regex duy nhất.

- Thuật toán tiêu biểu: Phương pháp loại trừ trạng thái hoặc thuật toán Arden.
- Ý nghĩa: Giúp con người đọc hiểu được logic của một hệ thống máy móc hoặc một mạch số phức tạp.

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

4. Ứng dụng thực tế

a) Trình tối ưu hóa Regex

Các bộ máy xử lý Regex hiện đại (như trong thư viện re của Python hay grep của Linux) sử dụng định lý Kleene để:

- ✱ Nhận một Regex từ người dùng.
- ✱ Chuyển nó sang NFA (theo chiều thuận).
- ✱ Chuyển NFA đó sang DFA để tìm kiếm với tốc độ cực nhanh.

b) Thiết kế và kiểm chứng phần cứng (Hardware Verification)

Khi thiết kế một con chip (là một hệ thống các trạng thái hữu hạn), các kỹ sư dùng định lý Kleene để trích xuất ra biểu thức chính quy tương đương với hành vi của chip đó. Điều này giúp họ kiểm tra xem con chip có hoạt động đúng theo đặc tả ban đầu hay không.

c) An ninh mạng và lọc dữ liệu

Các hệ thống phát hiện xâm nhập (IDS) mô tả các dấu hiệu tấn công bằng Regex. Nhờ định lý Kleene, các dấu hiệu này được nén lại thành các máy DFA cực kỳ tối ưu, cho phép quét hàng triệu gói tin mỗi giây mà không làm chậm mạng.

d) Lập trình trình biên dịch

Định lý Kleene là nền tảng cho các công cụ như Lex hoặc Flex. Nó giúp tự động hóa quá trình biến các quy tắc ngữ pháp của ngôn ngữ lập trình (như C, Java) thành các bộ máy quét từ vựng chạy trực tiếp trên máy tính.

Tóm tắt mối quan hệ giữa biểu thức chính quy, NFA, DFA:

Mô hình	Đặc điểm	Vai trò
Biểu thức chính quy	Đại số, khai báo	Mô tả (con người sử dụng)
NFA	Đồ thị, không đơn định	Thiết kế trung gian
DFA	Cấu trúc dữ liệu, đơn định	Thực thi (máy tính dùng)

Định lý Kleene chính là "chất keo" dính 3 mô hình này lại với nhau thành một hệ thống thống nhất.

BÀI TẬP BUỔI 4

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

GIÁO TRÌNH MÔ HÌNH TÍNH TOÁN

A little more persistence, a little more effort, and what seemed hopeless failure may turn to glorious success

Nếu bạn không muốn cố
gắng thì người khác muốn kéo
bạn! lên cũng không biết tay
bạn ở đâu!